

Introduction

L'objectif de ce projet est la conception et l'implémentation d'un moteur de recherche bibliothécaire complet, capable de traiter plus de 1600 ouvrages. Ce système ne se limite pas à une simple recherche par mots-clés ; il intègre une dimension structurelle et relationnelle inspirée des grands moteurs de recherche du web, en utilisant des algorithmes de théorie des graphes pour enrichir l'expérience utilisateur.

Problématique et Enjeux

Comment garantir une recherche à la fois rapide, flexible (via des expressions régulières) et pertinente dans un corpus de textes longs ? Pour répondre à cette question, notre solution s'appuie sur deux piliers :

1. **La pertinence structurelle** : Utilisation du score **PageRank** pour identifier les ouvrages "centraux" ou "de référence" au sein d'un graphe de similarités.
2. **La découverte de contenu** : Implémentation de l'indice de **Jaccard** pour calculer la proximité thématique entre les œuvres, permettant ainsi un système de recommandation dynamique ("Vous aimerez aussi").

Choix Technologiques

Le développement d'une telle solution nécessite une infrastructure robuste. Nous avons opté pour une architecture **micro-services conteneurisée via Docker**, permettant une isolation stricte des ressources. Le cœur du système est propulsé par **.NET 8**, choisi pour ses performances en calcul parallèle lors du traitement du graphe. La persistance des données est assurée par **PostgreSQL 15**, tandis que l'interface utilisateur est développée sous **Angular**, offrant une visualisation fluide et un surlignage dynamique des résultats.

Ce rapport détaille la mise en œuvre de ces algorithmes, de la phase critique d'acquisition des données à l'analyse de la complexité algorithmique, tout en évaluant les performances du système face aux contraintes réelles de ressources allouées. À travers l'étude de ce moteur de recherche, nous explorerons les compromis nécessaires entre précision théorique et efficacité logicielle.

1. Acquisition des données et constitution du Testbed

La validité des résultats d'un moteur de recherche dépend intrinsèquement de la qualité et de la structure des données sources. Pour ce projet, nous avons constitué un **testbed** (banc d'essai) représentatif de la littérature classique mondiale.

1.1 Provenance des données : Le Projet Gutenberg

Le choix s'est porté sur le **Projet Gutenberg**, la plus ancienne bibliothèque numérique au monde. Ce choix se justifie par trois facteurs :

- **Volume et diversité** : Accès à des milliers d'œuvres couvrant plusieurs siècles et langues.
- **Format brut** : La disponibilité des fichiers au format `.txt` (Plain Text UTF-8) permet d'éliminer les bruits liés au formatage (HTML/CSS) et de se concentrer sur l'analyse sémantique pure.
- **Libre de droits** : Permet une manipulation et une redistribution des données sans contraintes juridiques pour l'étude académique.

1.2 Méthode d'obtention : Le service GutenbergDownloader

L'acquisition n'est pas une simple copie de fichiers, mais un processus automatisé et sélectif implémenté dans le service GutenbergDownloader.

L'algorithme suit une logique de **moissonnage (scraping)** robuste :

1. **Itération itérative** : Le service parcourt les identifiants d'ouvrages. Compte tenu de la structure des URL du Projet Gutenberg (ex: `gutenberg.org/cache/epub/{id}/pg{id}.txt`), l'algorithme tente plusieurs miroirs pour maximiser le taux de succès.
2. **Gestion de la politesse** : Pour éviter de saturer les serveurs sources et de subir un bannissement d'IP, un délai (`Task.Delay`) est injecté toutes les 10 requêtes, respectant ainsi les bonnes pratiques du web scraping.
3. **Filtrage qualitatif** : L'un des points critiques de l'implémentation est le **seuil de volume**. Seuls les fichiers contenant plus de **10 000 mots** sont conservés. Ce critère élimine les pamphlets, les index ou les fichiers corrompus, garantissant que les algorithmes de similarité (Jaccard) et de classement (PageRank) travaillent sur des textes riches et comparables.

1.3 Analyse critique et gestion des erreurs

L'acquisition de données réelles sur le réseau introduit des instabilités que le code doit gérer. Notre service intègre une gestion d'exceptions granulaire :

- **Résilience HTTP** : Les codes d'erreur 404 (livre inexistant) ou 503 (serveur occupé) sont interceptés pour ne pas interrompre le processus global.
- **Critique du Testbed** : Bien que le seuil de 10 000 mots garantisse la qualité, il introduit un biais en faveur des romans au détriment de la poésie ou des essais courts. Cependant, pour un moteur de recherche basé sur la fréquence de termes, ce biais est un compromis nécessaire pour assurer la convergence statistique des algorithmes.

1.4 Pré-traitement : L'algorithme DataInitializer

Une fois le fichier brut téléchargé, il contient environ 15 à 20% de "bruit" (licences légales, préfaces de transpositeurs). Le service DataInitializer effectue un nettoyage par **expressions régulières (RegEx)**.

L'algorithme identifie les marqueurs standards :

- *** START OF THE PROJECT GUTENBERG EBOOK ***
- *** END OF THE PROJECT GUTENBERG EBOOK ***

Tout texte en dehors de ces balises est tronqué. Ensuite, l'algorithme extrait les métadonnées essentielles (Title, Author) par capture de motifs. Ce nettoyage est fondamental : sans lui, le PageRank identifierait indûment des relations de similarité entre tous les livres simplement parce qu'ils partagent la même licence légale en fin de fichier. Le résultat est une base de données "propre" où chaque entrée représente fidèlement l'œuvre littéraire originale.

2. Algorithme 1 : Similarité de Jaccard et Construction du Graphe

2.1 Définition du problème

Au sein d'une bibliothèque de plus de 1600 ouvrages, la recherche textuelle simple est limitée. Pour offrir une expérience de découverte, il est nécessaire de quantifier la **proximité thématique** entre deux manuscrits. Le problème consiste à transformer un contenu textuel non structuré en un ensemble de caractéristiques comparables afin de

générer un graphe où chaque nœud est un livre et chaque arête représente une ressemblance sémantique.

2.2 Fondement Théorique : L'indice de Jaccard

Pour mesurer cette similarité, nous utilisons l'indice de Jaccard. Contrairement à des mesures basées sur la fréquence brute, Jaccard compare la diversité des vocabulaires. Soient deux ensembles de mots A et B extraits de deux livres distincts, l'indice $J(A, B)$ est défini par le rapport entre la taille de l'intersection et la taille de l'union des ensembles :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

L'indice produit un score compris entre 0 (aucun mot commun) et 1 (vocabulaires identiques). Dans notre implémentation, un seuil de significativité (0.05) est appliqué pour ne conserver que les relations fortes et éviter la saturation du graphe.

2.3 Structures de données et Optimisation

Le calcul de la similarité entre chaque paire de livres présente une complexité algorithmique de $O(n^2)$. Pour 1664 livres, cela représente environ 1,38 million de comparaisons potentielles. Pour rendre ce traitement viable, deux structures de données clés ont été utilisées :

1. **Dictionary<int, HashSet<string>>** : Chaque livre est réduit à un HashSet de ses mots significatifs (après filtrage des *stop-words*). L'utilisation du HashSet permet des opérations d'intersection en temps quasi-constant $O(1)$ en moyenne.
2. **Index Inversé (Dictionary<string, List<int>>)** : Au lieu de comparer aveuglément chaque livre à tous les autres, nous utilisons un index inversé qui associe chaque mot à la liste des livres le contenant. Cela permet de n'identifier que les "candidats" ayant au moins un mot en commun avec le livre source, réduisant drastiquement le nombre de calculs inutiles.

2.4 Critique : Jaccard vs Similarité Cosinus

Dans la littérature du Traitement Automatique du Langage (NLP), la **similarité cosinus** est souvent privilégiée car elle prend en compte la fréquence des mots (TF-IDF). Cependant, dans le cadre de ce projet, l'indice de Jaccard a été préféré pour plusieurs raisons :

- **Sobriété calculatoire** : Jaccard sur des ensembles dédoublonnés est moins gourmand en mémoire que la manipulation de vecteurs de fréquences à haute dimension.
- **Nature des données** : Les livres du Projet Gutenberg sont de longueurs très inégales. La similarité cosinus peut parfois être biaisée par la longueur des documents, tandis que Jaccard, en se concentrant sur l'intersection du vocabulaire unique, offre une mesure de "parenté thématique" plus stable pour des ensembles de mots-clés extraits.
- **Efficacité des ensembles** : Le passage par des HashSet de mots-clés filtrés permet d'éliminer le bruit stylistique pour ne garder que la "signature" lexicale de l'œuvre.

3. Ranking via PageRank

3.1 Définition du problème : L'autorité au-delà de la pertinence

Une recherche par mots-clés classique renvoie souvent une liste de résultats où seule la fréquence du terme compte. Cependant, tous les ouvrages ne se valent pas. Le défi consiste à introduire une notion d'**autorité** ou de **centralité** : comment déterminer quels livres sont les plus "importants" au sein de la bibliothèque ?

En utilisant les relations de similarité calculées précédemment avec l'indice de Jaccard, nous traitons la bibliothèque comme un réseau interconnecté. Le problème revient alors à classer les nœuds de ce graphe non pas sur leur contenu intrinsèque, mais sur leur position relative dans la structure globale.

3.2 Fondement Théorique et Modélisation

L'algorithme PageRank, initialement conçu par Larry Page et Sergey Brin pour Google, repose sur l'idée qu'un lien (ou ici, une forte similarité) vers un nœud est un "vote de confiance". Le score d'un livre A dépend des scores de tous les livres B_n qui lui sont liés.

La formule itérative utilisée est la suivante :

$$PR(A) = \frac{1-d}{N} + d \sum_{i=1}^n \frac{PR(B_i)}{L(B_i)}$$

Où :

- $PR(A)$ est le score de l'ouvrage A.
- d est le **facteur d'amortissement**, fixé à **0.85**. Il représente la probabilité qu'un utilisateur continue sa navigation de livre en livre.
- N est le nombre total de livres.
- $L(B_i)$ est le nombre de liens sortants du livre B_i .

3.3 Structure de données et Implémentation

Pour manipuler ce graphe en mémoire, nous utilisons une **liste d'adjacence** optimisée via un `ConcurrentDictionary<int, List<int>>`. Cette structure permet d'accéder instantanément aux voisins d'un nœud lors de la phase de distribution des scores.

L'implémentation dans le service suit un processus rigoureux :

1. **Initialisation** : Chaque livre reçoit un score initial de $1/N$.
2. **Itérations** : Le calcul est répété **20 fois**. La littérature démontre que pour des graphes de cette taille, la convergence (stabilité des scores) est atteinte bien avant ce seuil.
3. **Distribution du "Link Juice"** : À chaque étape, chaque livre redistribue son score actuel équitablement entre ses voisins. Les livres "orphelins" (sans liens) voient leur score redistribué uniformément sur l'ensemble du graphe pour éviter les pertes d'énergie.

3.4 Analyse du code : Robustesse et Convergence

L'analyse de notre code révèle deux choix techniques majeurs :

- **Normalisation** : Après chaque itération, les scores sont recalculés pour s'assurer que leur somme totale reste égale à 1. Cela évite les dérives numériques et garantit la stabilité du système sous Docker, malgré les limites de précision des nombres à virgule flottante.
- **Performance** : Le calcul est effectué en mémoire vive (RAM) pour maximiser la vitesse, puis les résultats finaux sont persistés en une seule fois dans la colonne `PageRankScore` de la base PostgreSQL.

3.5 Critique : La pertinence du PageRank en bibliothèque

L'application du PageRank à la littérature, bien que non conventionnelle, offre des résultats fascinants :

- **Valorisation des Classiques** : Un ouvrage comme "Moby Dick" ou les œuvres de Shakespeare ont tendance à partager un vocabulaire riche et varié avec une

multitude d'autres livres. Par conséquent, ils deviennent des "hubs" dans notre graphe et obtiennent un PageRank naturellement élevé.

- **Amélioration du tri** : Lors d'une recherche sur un mot commun (ex: "Empire"), le moteur placera en tête les ouvrages de référence plutôt que des pamphlets obscurs contenant simplement le mot plus souvent.
- **Limites** : La principale critique réside dans la circularité. Si un groupe de livres médiocres est très similaire entre lui (copie ou thématique très étroite), ils peuvent artificiellement gonfler leur PageRank. C'est pourquoi le facteur d'amortissement de **0.85** est vital : il injecte une dose de "hasard" qui casse ces boucles fermées.

4. Algorithmes 3 : Moteur de Recherche Hybride

4.1 Définition du problème : Vitesse vs Flexibilité

Le défi d'un moteur de recherche moderne est de résoudre un paradoxe : offrir une puissance d'expression maximale (recherche par motifs complexes) tout en maintenant une latence imperceptible. Dans notre système, nous avons implémenté deux modes distincts pour répondre à deux besoins différents :

1. **La recherche simple** : Priorité à la rapidité et à l'autorité (PageRank).
2. **La recherche avancée (Regex)** : Priorité à la précision chirurgicale et à la puissance linguistique.

4.2 Architecture de la Recherche Simple

La recherche simple repose sur une stratégie de filtrage multicouche. L'objectif est de ne pas scanner l'intégralité des 1600 textes bruts en mémoire, ce qui paralyserait le processeur.

- **Filtrage SQL** : Nous utilisons une table d'indexation (IndexingTable) dans PostgreSQL. Une requête LIKE optimisée permet d'isoler en quelques millisecondes les candidats potentiels.
- **Calcul du score de pertinence** : Une fois les candidats extraits, un algorithme de "Scoring" pondéré est appliqué en C#. Le score final d'un livre pour une requête donnée est calculé ainsi :

$$Score_{final} = (W_{titre} \times M_{titre} + W_{contenu} \times M_{contenu}) \times PR_{score}$$

Où :

- $M_{titre/contenu}$: Nombre d'occurrences du mot-clé.
- $W_{titre/contenu}$: Poids accordés (généralement $W_{titre} \gg W_{contenu}$).
- PR_{score} : Le score **PageRank** calculé à l'étape précédente.

Cette formule garantit que les ouvrages les plus importants (autorités) remontent naturellement en tête de liste, même si un ouvrage mineur contient le mot-clé plus souvent.

4.3 Recherche Avancée : La puissance des Expressions Régulières

Le mode avancé permet d'utiliser des **RegEx**. C'est une fonctionnalité rare dans les moteurs de recherche grand public à cause de sa complexité algorithmique.

- **Définition théorique** : Une expression régulière est traduite en un automate fini. Sa complexité peut, dans certains cas pathologiques, devenir exponentielle ($O(2^n)$).
- **Gestion des ressources** : Pour éviter que le container Docker ne dépasse sa limite de 4GB de RAM, nous utilisons `IAsyncEnumerable`. Au lieu de charger les 1600 livres en une liste massive, le backend "stream" les livres depuis PostgreSQL, teste la RegEx, et renvoie le résultat au fil de l'eau.
- **Le Pipe de Surlignage (Angular)** : Le frontend reçoit non seulement le livre, mais aussi le motif recherché. Un `HighlightPipe` transforme dynamiquement le texte brut en HTML sécurisé en injectant des balises `<mark>`, permettant à l'utilisateur de visualiser instantanément le contexte de sa recherche.

4.4 Analyse du code et Optimisations

L'analyse de notre implémentation met en évidence deux optimisations majeures :

1. **Génération de Snippets** : Le moteur ne renvoie pas tout le livre (plusieurs Mo parfois) lors de la recherche, mais un "extrait" (snippet) entourant le premier match trouvé. Cela réduit drastiquement la charge réseau et le temps de rendu du navigateur.
2. **Encodage URI** : Pour le mode RegEx, les motifs complexes sont encodés (`encodeURIComponent`) avant d'être transmis à l'API, évitant les erreurs d'interprétation des caractères spéciaux par le protocole HTTP.

4.5 Critique : Les limites du "Full-Scan"

L'approche RegEx, bien que puissante, reste un "Full-Scan" linéaire. Contrairement à la recherche simple qui utilise un index, la RegEx oblige le CPU à tester le motif sur chaque caractère de chaque livre.

- **Amélioration possible** : Pour passer à l'échelle supérieure (100 000 livres), il faudrait implémenter des "Trigram Indexes" dans PostgreSQL, qui permettent d'accélérer certaines classes de RegEx.
- **Appréciation** : Pour un corpus de 1600 livres, notre architecture hybride offre le meilleur compromis : une recherche instantanée pour 95% des usages et une puissance de recherche experte pour les 5% restants.

5. Tests de Performance et Discussion

5.1 Protocole Expérimental

Pour évaluer la robustesse de la solution, les tests ont été effectués dans un environnement contrôlé via **Docker Desktop**, avec les limitations de ressources suivantes :

- **CPU** : 4.0 Virtual Cores (Host : Intel i5).
- **RAM** : 4 GB (limite critique pour le calcul du graphe en mémoire).
- **Base de données** : PostgreSQL 15-alpine sur volume SSD.
- **Échantillon** : 1664 ouvrages du Projet Gutenberg (env 1G de texte brut).

5.2 Analyse des temps de traitement pour l'initialisation

Le premier test porte sur le temps nécessaire pour passer de "zéro" à un moteur de recherche fonctionnel. Nous avons donc construit nos conteneurs en modifiant le nombre de livres à télécharger

Tableau de Synthèse des Performances

Nombre de Livres (N)	Taille Totale (Mo)	Ingestion DB (s)	Durée totale (minutes)
100	67.5	8.33	0.6
200	126.7	14.60	1.2
500	242.0	33.55	2

1664	780.7	149.75	10
------	-------	--------	----

Synthèse des Performances du Pipeline

L'analyse de la montée en charge sur quatre paliers (100 à 1664 livres) révèle deux dynamiques scalaires opposées :

- **Ingestion et Nettoyage ($O(N)$)** : Le traitement suit une progression **linéaire**. Le système maintient une cadence stable d'environ **85 ms/livre**, validant l'efficacité du nettoyage par Regex et de l'insertion SQL par lots.
- **Calcul du Graphe ($O(N^2)$)** : La phase de similarité (Jaccard) présente une croissance **quadratique**. Pour une augmentation du volume de x16, le temps de calcul est multiplié par **x100**. C'est le goulot d'étranglement du système, malgré l'optimisation par index inversé et le parallélisme CPU.
- **Calcul PageRank** : La performance est **quasi-instantanée** (< 1s pour 1664 nœuds). L'impact du classement sur le temps total est négligeable, confirmant la viabilité de l'algorithme pour des graphes de cette densité.

Verdict technique : Le système est hautement performant pour une bibliothèque de taille moyenne. Le passage à l'échelle supérieure (> 5000 livres) nécessiterait cependant une distribution du calcul de Jaccard sur plusieurs nœuds (Spark/Cloud) pour compenser l'explosion quadratique des comparaisons.

5.3 Latence des requêtes : Simple vs RegEx

La performance perçue par l'utilisateur dépend de la latence des endpoints de recherche. Nous avons mesuré le temps de réponse moyen (Time To First Byte) sur 100 requêtes aléatoires.

Type de requête	Succès	Temps moyen	Temps max	Observation
Recherche Simple	100/100	1 156 ms	2 855 ms	Excellente réactivité du moteur PostgreSQL.
Regex Basique	100/100	4 715 ms	9 966 ms	Pic de charge (Cold Start) détecté au début.
Regex Complexe	100/100	4 501 ms	5 457 ms	Performances stables une fois le moteur "chaud".

Analyse Technique

1. **Stabilité de l'infrastructure** : Le taux de succès de **100%** sur des requêtes intensives confirme que l'allocation de **4 vCPUs** et **4 Go de RAM** permet d'absorber la charge sans crash ni saturation de la mémoire
2. **Impact de la complexité** : Le passage de la recherche simple à la Regex multiplie le temps de réponse par environ **4**. Cela s'explique par le passage d'une recherche indexée (rapide) à un scan complet des colonnes par le moteur Regex de PostgreSQL, très gourmand en cycles processeur.
3. **Le phénomène du "Cold Start"** : Le temps maximum de **9 966 ms** (presque 10 secondes) sur la Regex basique indique un délai d'initialisation. C'est le moment où .NET compile les requêtes et où PostgreSQL prépare ses plans d'exécution en cache.
4. **Consistance des Regex** : Étonnamment, la Regex complexe est légèrement plus rapide et beaucoup plus stable que le basique. Cela démontre que le moteur gère très bien les motifs structurés une fois qu'il est sollicité de manière répétée.

7. Conclusion et Perspectives

7.1 Synthèse des travaux

Le projet **Gutenberg Search** a permis de démontrer qu'il est possible de transformer une bibliothèque numérique statique en un système de recherche dynamique et intelligent, même sous des contraintes de ressources matérielles strictes.

En combinant trois piliers algorithmiques — l'indice de **Jaccard** pour la proximité thématique, le **PageRank** pour la hiérarchisation par autorité, et les **Expressions Régulières** pour la flexibilité linguistique — nous avons abouti à une solution robuste. L'architecture micro-services sous Docker garantit la portabilité de la solution, tandis que le choix de la stack .NET/Angular assure une réactivité optimale de l'interface utilisateur.

7.2 Apports et limites

L'implémentation a mis en évidence l'importance du pré-traitement des données. Sans un nettoyage rigoureux des métadonnées légales du Projet Gutenberg, les scores de similarité auraient été artificiellement biaisés.

Cependant, le système rencontre ses limites dans le passage à l'échelle (scalabilité) :

- **Complexité temporelle** : Le calcul du graphe en $O(n^2)$ devient prohibitif au-delà de quelques milliers d'ouvrages.
- **Complexité spatiale** : Le stockage des ensembles de mots-clés en RAM limite la taille du corpus traitable sans architectures distribuées (type Spark).

7.3 Perspectives et évolutions futures

Pour dépasser ces limites et inscrire ce moteur dans les standards industriels actuels, plusieurs pistes d'évolution sont envisageables :

1. **Recherche Vectorielle (Semantic Search)** : Remplacer ou compléter l'indice de Jaccard par des *Embeddings* (vecteurs de sens) générés par des modèles comme BERT. Cela permettrait de trouver "Guerre et Paix" même si l'utilisateur tape "conflit napoléonien", grâce à la proximité sémantique dans un espace vectoriel.
2. **Indexation par Trigrammes** : Pour optimiser les recherches RegEx, l'utilisation de l'extension `pg_trgm` de PostgreSQL permettrait de pré-indexer les séquences de caractères et d'éviter les "full-scans" CPU-intensifs.
3. **Architecture Distribuée** : Migrer le calcul du PageRank vers un système de calcul distribué (type Orleans ou Akka.NET) pour traiter des millions d'ouvrages en répartissant la charge sur plusieurs nœuds de calcul.